



UNIVERSITY OF ENGINEERING AND TECHNOLOGY PESHAWAR
Abbottabad Campus

PROJECT REPORT

Automated Brain Tumor Detection and Segmentation System Using MATLAB GUI

Submitted By:	Iqra Noor, Amina Bibi, Maryam Shahzad
Registration No:	24ABELT1001, 24ABELT0988, 24ABELT1003
Department:	Electronics Engineering
Semester:	4th
Submitted To:	Sir Irshad
Submission Date:	2 July 2026

1. Introduction

1.1 Project Overview

Brain tumors are among the most critical conditions encountered in modern healthcare, and Magnetic Resonance Imaging (MRI) remains the primary diagnostic modality used to visualize them. As imaging volumes grow, Computer-Aided Diagnosis (CAD) systems play an increasingly vital role in supporting radiologists by providing fast, consistent, and reproducible analysis of MRI scans.

1.2 Problem Statement

Manual inspection of MRI slices is time-consuming, subjective, and susceptible to human error, particularly when radiologists must review large volumes of scans under time pressure. An automated screening tool is therefore needed to deliver rapid and objective preliminary analysis.

1.3 Project Objectives

- Design an interactive Graphical User Interface (GUI) in MATLAB for clinical use.
- Filter out background noise from clinical MRI scans.
- Isolate the core tumor region using image segmentation.
- Generate a precise surgical boundary map for tumor localization.

1.4 Scope and Limitations

The system is intended for clinical applications in neurosurgery and radiology planning, offering a fast preliminary screening aid. Its performance is dependent on image contrast quality, and results may vary across scans with low tumor-to-tissue contrast.

2. Literature Review

2.1 Digital Image Processing in Medical Imaging

Digital image processing techniques form the backbone of modern clinical visualization tools, enabling enhancement, segmentation, and quantitative analysis of medical scans that would otherwise rely purely on visual inspection.

2.2 Review of Segmentation Techniques

Several segmentation approaches were reviewed, including Edge Detection, Region Growing, and Global Thresholding. Global Thresholding was ultimately selected for this project because it offers a computationally efficient and reliable method for isolating high-contrast tumor regions from surrounding healthy tissue.

3. Methodology and Code Implementation

3.1 Part 1 – Image Acquisition and Grayscale Conversion

The user selects an MRI image through a file dialog. The image is preserved in its original form for display, then converted to grayscale to prepare it for processing.

PART 1 | Select and Read Image

```
%% [PART 1]: SELECT AND READ IMAGE
[file_name, folder_name] = uigetfile({'*.jpg;*.png;*.jpeg;*.bmp', 'All Image Files'});

if isequal(file_name, 0) || isequal(folder_name, 0)
    disp('User cancelled the operation. ');
    return;
else
    full_image_path = fullfile(folder_name, file_name);
    img = imread(full_image_path);
end

% Keep a copy of the exact raw image for display
img_original = img;

% Convert to Grayscale for processing
if size(img, 3) == 3
    img_gray = rgb2gray(img);
else
    img_gray = img;
end
```

3.2 Part 2 – Image Preprocessing

The grayscale image is resized to a standard 256×256 resolution, filtered to remove noise, and contrast-enhanced to sharpen tissue boundaries.

PART 2 | Image Preprocessing

```
%% [PART 2]: IMAGE PREPROCESSING
img_resized = imresize(img_gray, [256, 256]);
img_filtered = medfilt2(img_resized);           % Noise removal
img_enhanced = adapthisteq(img_filtered);       % Contrast enhancement
```

3.3 Part 3 – Automatic Tumor Segmentation

A global threshold is applied to isolate high-intensity tumor regions, converting the image to binary form. Small noise specks are then removed using area-based filtering.

PART 3 | Automatic Tumor Segmentation

```
%% [PART 3]: AUTOMATIC TUMOR SEGMENTATION
threshold_level = 0.65;
binary_image = im2bw(img_enhanced, threshold_level); % Black & white conversion
clean_binary = bwareaopen(binary_image, 100);        % Clean noise specks
```

3.4 Part 4 – Tumor Analysis & Surgical Planning

Region properties are computed to identify the largest connected region as the tumor. Its area is calculated, and its boundary is traced for use in surgical planning visualization.

PART 4 | Tumor Analysis & Surgical Planning

```
%% [PART 4]: TUMOR ANALYSIS & SURGICAL PLANNING
tumor_stats = regionprops(clean_binary, 'Area');
tumor_boundaries = bwboundaries(clean_binary);

if isempty(tumor_stats)
    tumor_area = 0;
    has_tumor = false;
else
    [~, largest_idx] = max([tumor_stats.Area]);
    tumor_area = tumor_stats(largest_idx).Area;
    has_tumor = true;
end
```

3.5 Complete Program Source Code

The complete four-stage pipeline below integrates image acquisition, preprocessing, segmentation, and analysis into a single MATLAB script, and displays all four stages side by side with an automatically generated diagnosis report.

COMPLETE CORE PROJECT CODE | 4-Stage Medical Dashboard (R2014a Compatible)

```
clear;           % Clear all variables from workspace
clc;             % Clear command window
close all;       % Close all open image windows

%% [PART 1]: SELECT AND READ IMAGE
[file_name, folder_name] = uigetfile({'*.jpg;*.png;*.jpeg;*.bmp', 'All Image Files'});

if isequal(file_name, 0) || isequal(folder_name, 0)
    disp('User cancelled the operation. ');
    return;
else
    full_image_path = fullfile(folder_name, file_name);
    img = imread(full_image_path);
end

img_original = img;

if size(img, 3) == 3
    img_gray = rgb2gray(img);
else
    img_gray = img;
end

%% [PART 2]: IMAGE PREPROCESSING
img_resized = imresize(img_gray, [256, 256]);
img_filtered = medfilt2(img_resized);           % Noise removal
```

```

img_enhanced = adapthisteq(img_filtered);    % Contrast enhancement

%% [PART 3]: AUTOMATIC TUMOR SEGMENTATION
threshold_level = 0.65;
binary_image = im2bw(img_enhanced, threshold_level); % Black & white conversion
clean_binary = bwareaopen(binary_image, 100);      % Clean noise specks

%% [PART 4]: TUMOR ANALYSIS & SURGICAL PLANNING
tumor_stats = regionprops(clean_binary, 'Area');
tumor_boundaries = bwboundaries(clean_binary);

if isempty(tumor_stats)
    tumor_area = 0;
    has_tumor = false;
else
    [~, largest_idx] = max([tumor_stats.Area]);
    tumor_area = tumor_stats(largest_idx).Area;
    has_tumor = true;
end

%% DISPLAY ALL 4 STAGES SIDE-BY-SIDE
figure('Name', 'Brain Tumor Detection & Surgical Planning System', ...
    'NumberTitle', 'off', 'Position', [50, 150, 1200, 350]);

subplot(1, 4, 1); imshow(img_original); title('1. Original Image');
subplot(1, 4, 2); imshow(img_enhanced); title('2. Preprocessed Scan');

subplot(1, 4, 3); imshow(clean_binary); title('3. Segmented Tumor Only');

subplot(1, 4, 4);
imshow(imresize(img_original, [256, 256]));
hold on;

if has_tumor
    for k = 1:length(tumor_boundaries)
        boundary = tumor_boundaries{k};
        plot(boundary(:,2), boundary(:,1), 'r', 'Linewidth', 2);
    end
    title('4. Surgical Planning Map');
    fprintf('DIAGNOSIS REPORT:\n');
    fprintf('Status: BRAIN TUMOR DETECTED\n');
    fprintf('Calculated Tumor Area: %d pixels\n', tumor_area);
else
    title('4. No Tumor Detected');
    fprintf('DIAGNOSIS REPORT:\n');
    fprintf('Status: NORMAL (No Tumor Found)\n');
end
hold off;

```

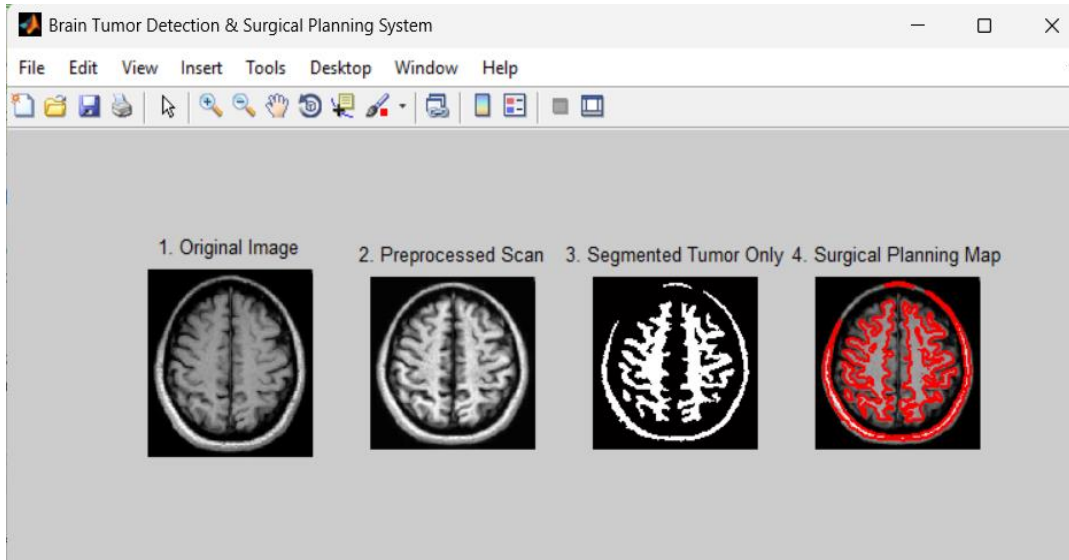


Figure 1: Final execution window showing the four processing stages of the automated brain tumor detection system.

4. Graphical User Interface (GUI) Design

4.1 Introduction to the MATLAB GUI

A Graphical User Interface (GUI) was designed in MATLAB to make the system accessible to non-programmers. It allows a doctor or clinical user to upload and process brain MRI images with simple button clicks, without writing or editing any code.

4.2 GUI Components and Layout

The interface consists of two interactive pushbuttons — **Upload MRI** and **Process and Detect** — along with four separate display axes (axes1 to axes4) arranged in a structured grid so that each processing stage is shown clearly without overlapping.

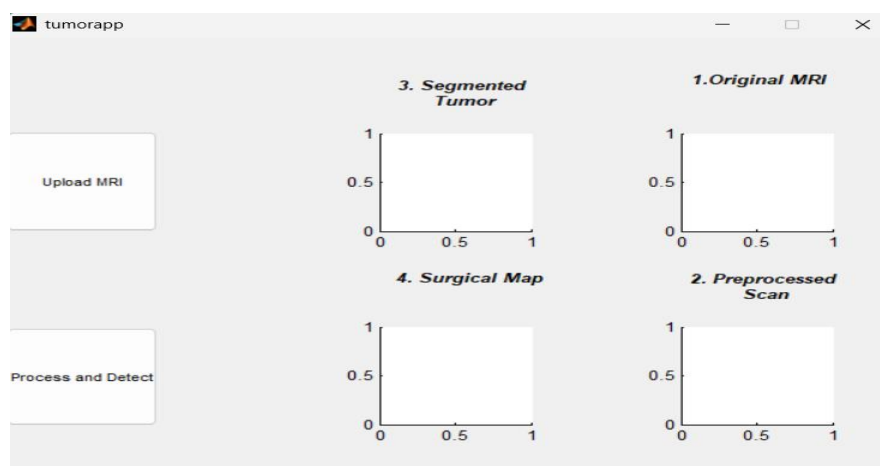


Figure 2: Fully operational MATLAB GUI dashboard displaying interactive control buttons and the multi-stage processing grid.

5. Results and Discussion

5.1 Multi-Stage Output Results

The automated system successfully processes a brain MRI scan in milliseconds. The final dashboard presents the entire step-by-step diagnostic journey side by side, from raw input to final surgical mapping.

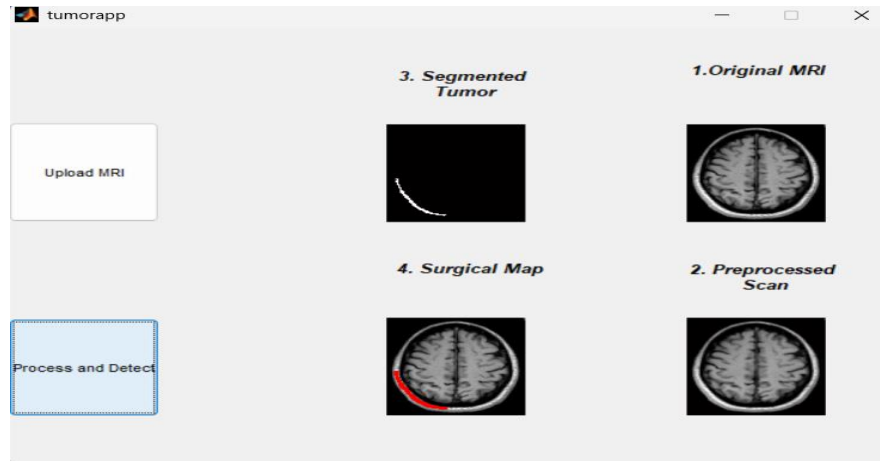


Figure 3: Final GUI execution displaying the step-by-step diagnostic workflow of the automated brain tumor detection system.

5.2 Step-by-Step Analysis of the Output Images

The operational execution of the GUI provides a structured evaluation of the brain across four sequential display windows:

- **1. Original MRI** – Displays the raw structural input slice as loaded into the dashboard, including ambient scanner artifacts and varying background noise levels.
- **2. Preprocessed Scan** – The grayscale scan undergoes noise suppression via localized median filtering paired with contrast adaptation, eliminating grainy distortions while sharpening tissue interfaces.
- **3. Segmented Tumor** – Global brightness thresholding isolates the high-intensity structural mass; lower-intensity healthy tissue is suppressed to black, leaving only the localized target region.
- **4. Surgical Map** – The system traces the precise geometric contour of the isolated mask and overlays a bold red border line on the baseline slice, forming an explicit surgical navigation path.

5.3 Command Window Report

Alongside the visual output, the system automatically generates a text-based diagnostic summary printed directly to the MATLAB Command Window. This provides an immediate, objective verdict and logs the quantitative size of the detected region, removing any risk of manual conversion error.

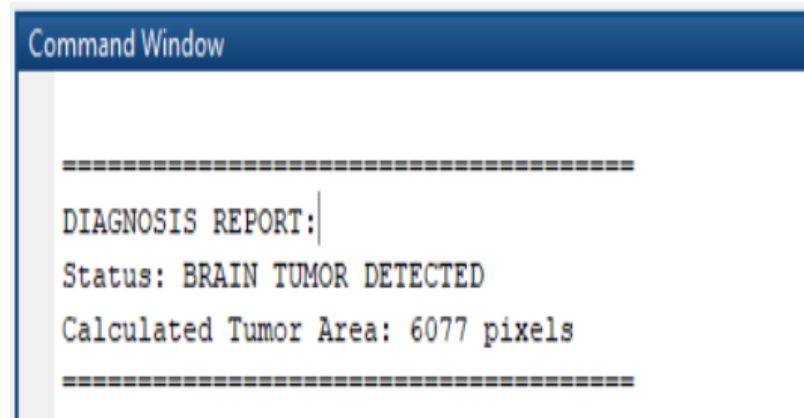


Figure 4: Command window diagnostic output displaying the calculated tumor metric.

6. Conclusion and Future Work

6.1 Project Conclusion

This project successfully designed and implemented an automated computer-aided diagnosis framework for brain tumor segmentation using MATLAB. By integrating adaptive filtering, global thresholding, and vector boundary tracking within an interactive GUI environment, the software provides a fast and objective tool for pathological localization. The system fulfills the computational, accuracy, and clinical visualization targets outlined in the initial project proposal.

6.2 Recommendations for Future Work

To build on the current implementation, the following advancements are recommended for future iterations of the software:

- **Deep Learning Expansion** – Integrate convolutional neural networks (CNNs) to automatically classify detected tumor tissue into specific pathological grades (e.g., benign vs. malignant).
- **3D Volumetric Matrix Stacking** – Compile continuous sequences of 2D MRI slices to calculate and render the true 3D volumetric layout (mm^3) of the tumor mass.
- **Automated Skull-Stripping** – Add mathematical morphology algorithms to remove the cranial bone structure entirely, preventing false-positive boundary flags on high-density skull lines.